

Pilha, Fila e Struct

Questão 1 · Tarefas com prioridade

Enunciado

Considere a struct que representa uma tarefa: id (int), descricao (char[50]) e prioridade (1 = alta, 2 = média, 3 = baixa).

a) Implemente uma pilha de tarefas. b) Implemente também uma fila de tarefas. c) Crie um programa que leia 5 tarefas e insira as de prioridade alta (1) na pilha e as de prioridade média ou baixa (2 ou 3) na fila. Ao final, exiba as tarefas da pilha e as da fila.

Explicação passo a passo

Em C, uma estrutura de dados é uma fila de **caixinhas** (nós). Cada caixinha guarda um valor (aqui, uma **Tarefa**) e uma **setinha** (ponteiro) que aponta para a próxima caixinha. Quase tudo o que fazemos é mexer nessas setinhas.

a) Pilha: uma pilha de pratos

O último prato colocado é o primeiro a ser retirado. Isso se chama **LIFO** (Last In, First Out).

- **push** (empilhar): cria uma caixinha nova, aponta a setinha dela para o antigo topo e a torna o novo topo.
- **pop** (desempilhar): tira a caixinha do topo e o topo passa a ser a próxima.
- **exibir**: passeia do topo até a base, sem retirar nada.

b) Fila: a fila do pão

Quem chega primeiro é atendido primeiro. Isso se chama **FIFO** (First In, First Out).

- **enfileirar**: coloca a caixinha nova no fim da fila.
- **desenfileirar**: tira a caixinha do início.
- A fila guarda dois dedos: um no início e outro no fim. Assim, entrar na fila é rápido, sem precisar andar até o final.

c) Programa principal

- 1 Repete 5 vezes: lê o id, a descrição e a prioridade (repetindo a pergunta se a prioridade não for 1, 2 ou 3).
- 2 Se a prioridade for 1, a tarefa vai para a **pilha** (push). Se for 2 ou 3, vai para a **fila** (enfileirar).
- 3 No final, exibe a pilha e a fila, e libera a memória.

Detalhe do scanf: a descrição é lida com " %49[^\n]", que lê a frase inteira com espaços ("Entregar trabalho") e no máximo 49 caracteres, cabendo no vetor de 50. Com %s, a leitura pararia no primeiro espaço.

Código completo

Arquivo: 01_pilha_fila_struct.c. Para compilar: gcc -Wall -o programa 01_pilha_fila_struct.c && ./programa

```
/* Pilha, Fila e Struct: tarefas */
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id;
    char descricao[50];
    int prioridade; /* 1 = alta, 2 = média, 3 = baixa */
} Tarefa;

/* Cada "caixinha" da pilha/fila guarda uma Tarefa e aponta para a próxima */
typedef struct NoTarefa {
    Tarefa tarefa;
    struct NoTarefa* prox;
} NoTarefa;

/* ----- a) PILHA (LIFO: o último que entra é o primeiro que sai) ----- */
typedef struct { NoTarefa* topo; } Pilha;

void pilha_iniciar(Pilha* p) { p->topo = NULL; }

int pilha_push(Pilha* p, Tarefa t) {
    NoTarefa* novo = (NoTarefa*) malloc(sizeof(NoTarefa));
    if (!novo) return 0;
    novo->tarefa = t;
    novo->prox = p->topo; /* o novo aponta para quem era o topo */
    p->topo = novo; /* e vira o novo topo */
    return 1;
}

int pilha_pop(Pilha* p, Tarefa* saida) {
    if (!p->topo) return 0;
    NoTarefa* rem = p->topo;
    *saida = rem->tarefa;
    p->topo = rem->prox;
    free(rem);
    return 1;
}

void pilha_exibir(Pilha* p) {
    printf("=== PILHA (topo -> base) ===\n");
    if (!p->topo) printf("(vazia)\n");
    for (NoTarefa* n = p->topo; n; n = n->prox)
        printf("ID: %d | %s | Prioridade: %d\n", n->tarefa.id, n->tarefa.descricao, n->tarefa.prioridade);
}

void pilha_liberar(Pilha* p) { Tarefa t; while (pilha_pop(p, &t)); }

/* ----- b) FILA (FIFO: o primeiro que entra é o primeiro que sai) ----- */
typedef struct { NoTarefa* inicio; NoTarefa* fim; } Fila;

void fila_iniciar(Fila* f) { f->inicio = f->fim = NULL; }

int fila_enfileirar(Fila* f, Tarefa t) {
    NoTarefa* novo = (NoTarefa*) malloc(sizeof(NoTarefa));
    if (!novo) return 0;
    novo->tarefa = t;
    novo->prox = NULL;
    if (f->fim) f->fim->prox = novo; else f->inicio = novo;
    f->fim = novo;
    return 1;
}
```

```

int fila_desenfileirar(Fila* f, Tarefa* saida) {
    if (!f->inicio) return 0;
    NoTarefa* rem = f->inicio;
    *saida = rem->tarefa;
    f->inicio = rem->prox;
    if (!f->inicio) f->fim = NULL;
    free(rem);
    return 1;
}

void fila_exibir(Fila* f) {
    printf("=== FILA (inicio -> fim) ===\n");
    if (!f->inicio) printf("(vazia)\n");
    for (NoTarefa* n = f->inicio; n; n = n->prox)
        printf("ID: %d | %s | Prioridade: %d\n", n->tarefa.id, n->tarefa.descricao, n->tarefa.prioridade);
}

void fila_liberar(Fila* f) { Tarefa t; while (fila_desenfileirar(f, &t)); }

/* ----- c) Programa principal ----- */
int main(void) {
    Pilha pilha; Fila fila;
    pilha_iniciar(&pilha);
    fila_iniciar(&fila);

    for (int i = 1; i <= 5; i++) {
        Tarefa t;
        printf("\n--- Tarefa %d de 5 ---\n", i);
        printf("ID: ");
        scanf("%d", &t.id);
        printf("Descricao: ");
        scanf(" %49[^\n]", t.descricao); /* lê a frase inteira, com espaços */
        do {
            printf("Prioridade (1=alta, 2=media, 3=baixa): ");
            scanf("%d", &t.prioridade);
        } while (t.prioridade < 1 || t.prioridade > 3);

        if (t.prioridade == 1) pilha_push(&pilha, t);
        else fila_enfileirar(&fila, t);
    }

    printf("\n");
    pilha_exibir(&pilha);
    printf("\n");
    fila_exibir(&fila);

    pilha_liberar(&pilha);
    fila_liberar(&fila);
    return 0;
}

```

Teste executado

Entradas: (1, Lavar louca, 2), (2, Estudar C, 1), (3, Fazer compras, 3), (4, Entregar trabalho, 1), (5, Ler livro, 2). A pilha mostra o ID 4 primeiro porque foi o último a entrar.

```

=== PILHA (topo -> base) ===
ID: 4 | Entregar trabalho | Prioridade: 1
ID: 2 | Estudar C | Prioridade: 1

```

```

=== FILA (inicio -> fim) ===
ID: 1 | Lavar louca | Prioridade: 2
ID: 3 | Fazer compras | Prioridade: 3
ID: 5 | Ler livro | Prioridade: 2

```